

Lec 5

Software Security Design

Chapter 8 Principles of Security Models, Design, and Capabilities

“CISSP Certified Information Systems Security Professional Official Study Guide,” 9th ed.
by Mike Chapple, James M Stewart, Darril Gibson, Sybex, 2021

Outline

- Principles of Security Models, Design, and Capabilities (CISSP book: ch 8)
 - Secure design principles
 - Techniques for ensuring CIA
 - Fundamentals concepts of security models
 - Understand Security Capabilities of Information Systems
- Security design patterns

Secure Design Principles

- **Security** should be a consideration **at every stage** of a system's development.
- Developers should strive to **build security into every application** they develop,
 - with greater levels of security provided to critical applications and those that process sensitive information.
- It's extremely important to consider the security implications of a development project from the **early stages**
 - because it's much easier to build security into a system than it is to add security onto an existing system.
- Developers should research, implement, and manage engineering processes using **secure design principles**

Secure Design Principles

CISSP	US-CERT
- Fail Securely	- Failing Securely
- Defense in Depth (layering)	- Defense in Depth
- Least Privilege	- Least Privilege
- Separation of Duties	- Separation of Privilege
- Privacy by Design	- Promoting Privacy
- Zero Trust	- Reluctance to Trust
- Keep It Simple	- Economy of Mechanism
- Trust but Verify	- Least Common Mechanism
- Secure Defaults	- Never Assuming that your Secrets are Safe
-----> more on chapter 16	- Complete Mediation
	- Psychological Acceptability
	- Securing the Weakest Link

Review: Objects and Subjects

- Subject
 - user or process that makes a request to access a resource.
Access can mean reading from or writing to a resource.
- Object
 - resource a user or process wants to access

(Keep in mind that the subject and object refer to some specific access request, so the same resource can serve as a subject and an object in different access requests.)

Review: Objects and Subjects

- Process A may ask for data from process B.
 - To satisfy process A's request, process B must ask for data from process C.
-
- In this example, process B is the object of the first request and the subject of the second request:

First request	process A (subject)	process B (object)
Second request	process B (subject)	process C (object)

Terminology: Transitive Trust

- *Transitive trust* is the concept that if A trusts B and B trusts C, then A inherits trust of C through the transitive property
- Transitive trust is a serious security concern
 - because it may enable bypassing of restrictions or limitations between A and C, especially if A and C both support interaction with B.
- An example : an organization blocks access to Facebook or YouTube to increase worker productivity.
 - Thus, workers (A) do not have access to certain internet sites (C).
 - However, if workers can access a web proxy, virtual private network (VPN), or anonymization service, then this can serve as a means to bypass the local network restriction.
 - If workers (A) are accessing VPN service (B), and the VPN service (B) can access the blocked internet service (C); then A is able to access C through B via a **transitive trust exploitation**.

Terminology: Closed and Open Systems

Systems are designed and built according to one of two differing philosophies:

- A *closed system* is designed to work well with a narrow range of other systems, generally all from the same manufacturer.
 - The standards for closed systems are often proprietary and not normally disclosed.
 - Closed systems are harder to integrate with unlike systems, but they can be more secure.
- *Open systems* are designed using agreed-upon industry standards.
 - Open systems are much easier to integrate with systems from different manufacturers that support the same standards.
 - More vulnerable to attack

Secure Defaults

- Often the default settings of a product are selected to minimize installation problems to avoid increased load on the technical support services.
 - Thus, never assume that the default settings of any product are secure
 - Should assume that default settings are the worst options
 - need to review every setting and configure to optimize security
- there is some movement toward more secure defaults.
 - Microsoft uses a **Security Development Lifecycle (SDL)**
 - “Secure by Design, Secure by Default, Secure in Deployment and Communication” (also known as **SD3+C**).
 -
- Some products, especially security products, may now be designed with their most secure settings enabled by default
 - But may need good documentation to explain configuration

Fail Securely

- **System failures** can occur due to a wide range of causes.
 - Once fail, how system / environment handles failure is important.
 - The most desired result is for an application to **fail securely**.
- Failure management
 - Exception handling (for programmatic error)
 - Input sanitization, input filtering, or input validation (for bad input)

Fail Securely

- several similar terms
 - Fail-soft, fail-secure, fail-open, fail-closed
- Fail-soft
 - allow a system to continue to operate after a component fails
 - an alternative to having a failure cause a complete system failure.
 - Example: a typical multitasking operating system that can support numerous simultaneous applications. If one application fails, the others can typically continue to operate.

Fail Securely

- Fail-open
 - the priority is for maintaining availability
 - when the product fails, the connection or communication is allowed to continue (maybe without filtering)
- Fail-closed
 - the priority is for maintaining confidentiality and integrity
 - when the product fails, the connection or communication is cut off
 - Also known as fail-secure, and/or fail-safe
 - Ex. when an operating system encounters a processing or memory isolation violation,
 - it terminates all executions, then initiates a reboot
 - known as a stop error, or the Blue Screen of Death (BSOD) in Windows.

Fail Securely

- Fail-safe
 - Focus on protecting the health and safety of people
 - example, a fail-safe door will open easily in an emergency in order to allow people to escape a building
 - the protection of assets may be sacrificed
- Fail-secure
 - Focus on protecting physical security of assets
 - example, a vault door may automatically close and lock when the building enters a state of emergency
 - the potential cost of harming personnel who could be trapped inside

TABLE 8.2 Fail terms definitions related to physical and digital products

Physical	State	Digital
Protect People	Fail-Open	Protect Availability
Protect People	Fail-Safe	Protect Confidentiality & Integrity
Protect Assets	Fail-Closed	Protect Confidentiality & Integrity
Protect Assets	Fail-Secure	Protect Confidentiality & Integrity

Keep It Simple

- a shortened form of the classic statement of “keep it simple, stupid”
 - (the **KISS principle**)
- In security, we should avoid overcomplicating the environment, organization, or product design.
 - The more complex a system, the more difficult it is to secure.
- Related concepts
 - **“Don't Repeat Yourself” (DRY)**
 - **Rule of Least Power** Use the least powerful programming language that is suitable for the needed solution.
 - **“You Aren't Gonna Need It” (YAGNI)** Programmers should not add capabilities and functions until they are actually necessary

Zero Trust

- Zero trust is a security concept where nothing inside the organization is automatically trusted.
 - Before: the inside is trusted, the outside is untrusted
 - Now: both inside and outside are untrusted
 - Since now inside can mean mobile devices, clouds, IoTs, etc.
- There have been too many occurrences of security breaches caused by insiders
- Thus, every transaction should be verified before it is allowed to occur.
 - internal segmentation firewalls, multi-factor authentication, identity and access management
- The goal is to have every access request be authenticated, authorized, and encrypted prior to the access being granted to a resource or asset.

Privacy by Design

- Privacy by Design (PbD) is a guideline to
 - **integrate privacy** protections into products during the **early design phase**
 - rather than attempting to tack it on at the end of development.
- same overall concept as “security by design” or “integrated security,”
- The goal of PbD is to have developers integrate privacy protections into their solutions in order to avoid privacy violations in the first place
 - preventions rather than remedies for violations.

Trust but Verify

- Often the trust but verify approach depended on an initial authentication process to gain access to the internal “secured” environment, and then relied on generic access control methods.
- Due to the rapid growth and changes in the modern threatscape, the trust but verify model of security is no longer sufficient. Most security experts now recommend designing organizational security around the zero trust model.

Defense in Depth

- Layering in Lec 1
- use of multiple controls **in a series**.
- **No one** control can protect against **all** possible threats.
- Using a **multilayered** solution allows for numerous, different controls to guard against whatever threats come to pass.

Least Privilege¹

- A subject should be given only those privileges that it needs in order to complete its task.
 - Default should be lack of access
 - Elevate the right only when needed
 - And rights should be removed as soon as those rights are no longer required.

e.g.

sudo
su programs
set uid only when needed

¹ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/least-privilege>

Separation of Duties

- no single person has total control over a critical function or system.
 - ensure that no single person can compromise the system or its security
- two or more people must conspire or collude against the organization, which increases the risk for these people.
 - makes it more difficult for individuals to engage in malicious, fraudulent, or unauthorized activities
- dividing the security or administrative capabilities and functions among multiple trusted individuals.
 - no single person has sufficient access to circumvent or disable security mechanisms.
 - people only have the privileges they need to do their job,
 - make sure their job isn't too big. When someone does a job that's too big, they'll need lots of permissions to do that job

Separation of Privilege²

- A system should not grant permission based on a single condition.
 - The principle is similar to the separation of duty principle.
 - Thus before privilege is granted two or more checks should be performed.

e.g.

to su (change) to root two conditions must be met

1. the user must know the root password
2. the user must be in the right group (wheel)

² <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/separation-of-privilege>

Fail-Safe Defaults³

- Unless a subject is given explicit access to an object, it should be denied access to that object.
 - Deny by default
 - Need to argue why a user should have access. Do not argue why a user should not have access
 - If action fails, system as secure as when action began

³ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/failing-securely>

Economy of Mechanism⁴

- Security mechanisms should be as simple as possible.
 - Simpler means less can go wrong
 - If the security design and implementation are simple, fewer possibilities exist for errors
 - checking/testing process is less complex, easier to understand and fix
 - Interfaces between security modules are suspect areas and should be as simple as possible.

⁴ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/economy-of-mechanism>

Complete Mediation⁵

- All accesses to objects should be checked to ensure that they are allowed.
 - Every access to every object must be checked / authenticated
 - Must be efficient
 - In addition to normal runtime, must be done at:
 - Initialization
 - shutdown
 - Restart

⁵ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/complete-mediation>

Open Design⁶

- The security of a mechanism should not depend on the secrecy of its design or implementation.
 - This principle suggests that complexity does not add security.
 - “Security through obscurity” is a bad idea
 - Should be open for scrutiny by the community
 - Better to have a friend/colleague find an error than a foe

⁶ <https://github.com/OWASP/DevGuide/blob/master/02-Design/01-Principles%20of%20Security%20Engineering.md#open-design>

Least Common Mechanism⁷

- Mechanisms used to access resources should not be shared.
 - Minimize the amount of mechanism common to more than one user
 - Every shared mechanism is a potential information path
- Hence, do not share objects and protection mechanisms, instead create separate instances for each user or system interface.

⁷ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/least-common-mechanism>

Psychological Acceptability⁸

- Security mechanisms should not make the resource more difficult to access than if the security mechanism were not present
 - If security-related software or systems are too complicated to use, the user will not employ the security mechanisms.
 - The security mechanism may add some extra burden, but that burden must be both minimal and reasonable.
 - At the same time, programs should not impart unnecessary information that may lead to a compromise in security.

e.g.

When you enter a wrong password, the system should only tell you that the user id or password was wrong. It should not tell you that only the password was wrong as this gives the attacker information.

⁸ <https://us-cert.cisa.gov/bsi/articles/knowledge/principles/psychological-acceptability>

Outline

- Principles of Security Models, Design, and Capabilities (CISSP book: ch 8)
 - Secure design principles
 - Techniques for ensuring CIA
 - Fundamentals concepts of security models
 - Understand Security Capabilities of Information Systems
- Security design patterns

Techniques for Ensuring CIA

- Confinement
- Bounds
- Isolation
- Access Controls
- Trust and Assurance

Confinement

- restrict the actions of a program to specific locations and resources
 - process can read from and write to only certain memory locations and resources.
 - the application of the principle of least privilege to processes.
- Also known as **Sandboxing**
- The goal of confinement
 - prevent data leakage to unauthorized programs, users, or systems.

Bounds

- Each process that runs on a system is assigned an authority level.
- Authority level
 - Ex. two authority levels: user and kernel
- The authority level tells the operating system how to set the **bounds** for a process
- The bounds of a process consist of
 - limits set on the memory addresses and resources it can access.
- Bounds can be a means to enforce confinement.

Isolation

- When a process is confined through enforcing access bounds, that process runs in isolation
- Isolation is what prevents an application from accessing the memory or resources of another application

Access Controls

To ensure the security of a system, you need to allow subjects to access only authorized objects.

- Access controls limit the access of a subject to an object.
- One common control is for file access.
 - A file can be protected from modification by making it read-only for most users
 - but read-write for a small set of users who have the authority to modify it.

Trust and Assurance

A trusted system is one in which all protection mechanisms work together to process sensitive data for many types of users

- while maintaining a stable and secure computing environment.
- trust is the presence of a security mechanism, function, or capability.

Assurance is the degree of confidence in satisfaction of security needs.

- assurance is how reliable the security mechanisms are at providing security.
- Assurance must be continually maintained, updated, and reverified.
- Assurance varies from one system to another and often must be established on individual systems

Trust can be built into a system by implementing specific security features,
Assurance is an assessment of the reliability and usability of those security features in a real-world situation.

Outline

- Principles of Security Models, Design, and Capabilities (CISSP book: ch 8)
 - Secure design principles
 - Techniques for ensuring CIA
 - Fundamentals concepts of security models
 - Understand Security Capabilities of Information Systems
- Security design patterns

Fundamental Concepts of Security Models

- A *security model* provides a way for designers to map abstract statements into a security policy that prescribes the algorithms and data structures necessary to build hardware and software.
- These models offer a way to deepen your understanding of how a system should be designed and developed to support a specific security policy.
 - gives software designers something against which to measure their design and implementation.

Security Models

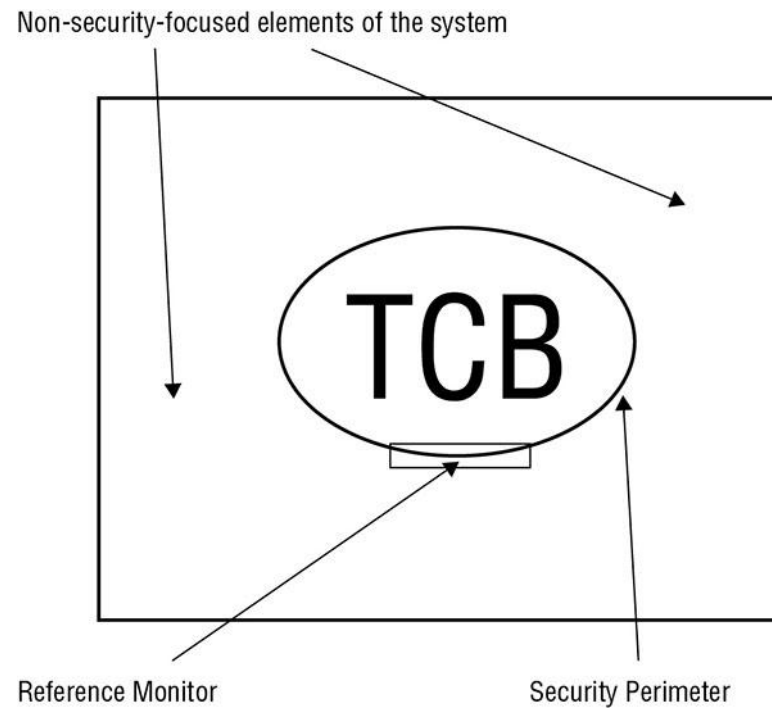
Security model can shed light on how security enters into computer architectures and operating system design:

- Trusted computing base
- State machine model
- Information flow model
- Noninterference model
- Take-grant model
- Access control matrix
- Bell–LaPadula model
- Biba model
- Clark–Wilson model
- Brewer and Nash model
- Goguen–Meseguer model
- Sutherland model
- Graham–Denning model
- Harrison–Ruzzo–Ullman model

TRUSTED COMPUTING BASE

- Trusted computing base.
 - a foundation on which most security models are built
- a combination of hardware, software, and controls that work together to form a trusted base to enforce your security policy.
- The TCB is a subset of a complete information system that can be trusted to adhere to and enforce the security policy.
 - should be as small as possible so that a detailed analysis can reasonably ensure that the system meets design specifications and requirements.
 - The TCB is the only portion of that system that can be trusted to adhere to and enforce the security policy.

TCB: Security Perimeter



- The security perimeter of your system is an imaginary boundary that separates the TCB from the rest of the system

TCB: Security Perimeter

- This boundary ensures that no insecure communications or interactions occur between the TCB and the remaining elements of the computer system.
- For the TCB to communicate with the rest of the system, it must create secure channels, also called *trusted paths*.
- A trusted path is a channel established with strict standards to allow necessary communication to occur without exposing the TCB to security vulnerabilities.
 - A trusted path also protects system users (sometimes known as subjects) from compromise as a result of a TCB interchange.

TCB: Reference Monitors and Kernels

- The reference monitor validates access to every resource prior to granting access requests
- the *security kernel* are collection of components in the TCB that work together to implement reference monitor functions and resist all known attacks.

State Machine Model

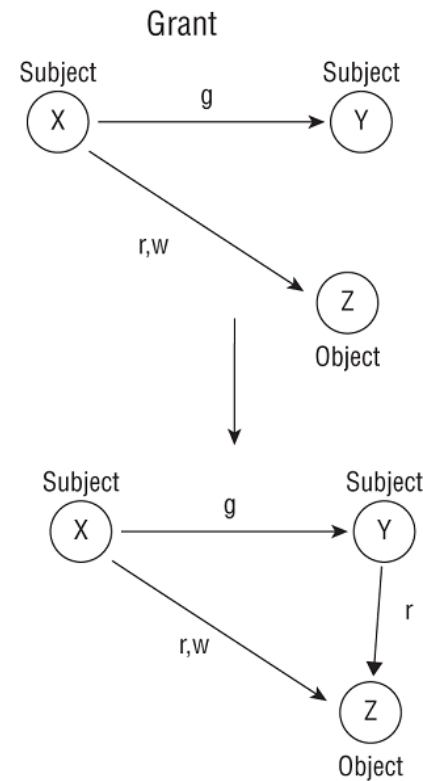
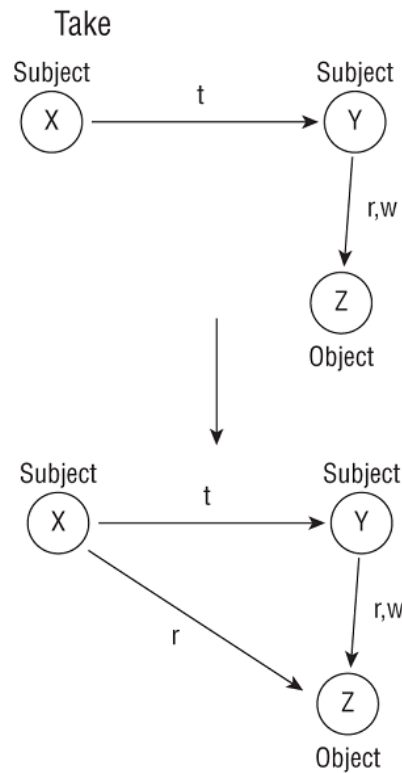
- The *state machine model* describes a system that is always secure no matter what state it is in.
- It's based on the computer science definition of a *finite state machine (FSM)*.
- Many security models are based on the secure state concept.
 - If all aspects of a state meet the requirements of the security policy, that state is considered secure.
 - All state transitions must be evaluated.
- If each possible state transition results in another secure state, the system can be called a *secure state machine*.
- The secure state machine model is the basis for many other security models.

Information Flow Model

- The *information flow model* focuses on the flow of information.
 - Information flow models are based on a state machine model.
- Information flow models don't necessarily deal with only the direction of information flow; they can also address the type of flow.
- Information flow models are designed to prevent unauthorized, insecure, or restricted information flow, often between different levels of security (these are often referred to as multilevel models).
 - Information flow can be between subjects and objects at the same classification level as well as between subjects and objects at different classification levels.
 - allows all authorized information flows, whether within the same classification level or between classification levels.
 - prevents all unauthorized information flows, whether within the same classification level or between classification levels

Take-Grant Model

- The *Take-Grant model* describes how rights can be passed from one subject to another or from a subject to an object.
 - A subject X with the grant right can grant another subject Y or another object Z any other right that X possesses.
 - A subject X with the take right can take a right from another subject Y.
- The key to this model is that using these rules allows you to figure out when rights in the system can change and where leakage (that is, unintentional distribution of permissions) can occur.



- In essence, here are the four rules of the take-grant model:
 - Take rule: Allows a subject to take rights over an object
 - Grant rule: Allows a subject to grant rights to an object
 - Create rule: Allows a subject to create new rights
 - Remove rule: Allows a subject to remove rights it has

Access Control Matrix

- An *access control matrix* is a table of subjects and objects that indicates the actions or functions that each subject can perform on each object.
- Each column of the matrix is an *access control list (ACL)*.
- Each row of the matrix is a *capabilities list*.
 - An ACL is tied to the object; it lists valid actions each subject can perform.
 - A capability list is tied to the subject; it lists valid actions that can be taken on each object.

Example: An access control matrix

The access control matrix is for a discretionary access control system. A mandatory or rule-based matrix can be constructed simply by replacing the subject names with classifications or roles.

Subjects	Document file	Printer	Network folder share
Bob	Read	No Access	No Access
Mary	No Access	No Access	Read
Amanda	Read, Write	Print	No Access
Mark	Read, Write	Print	Read, Write
Kathryn	Read, Write	Print, Manage Print Queue	Read, Write, Execute
Colin	Read, Write, Change Permissions	Print, Manage Print Queue, Change Permissions	Read, Write, Execute, Change Permissions

Bell– LaPadula Model

The U.S. Department of Defense (DoD) developed the *Bell-LaPadula model* in the 1970s to address concerns about **protecting classified information.**

- By design, the Bell-LaPadula model prevents the leaking or transfer of classified information to less secure clearance levels.
 - focused on maintaining the confidentiality of objects.
 - does not address the aspects of integrity or availability for objects.
- This is accomplished by blocking lower-classified subjects from accessing higher-classified objects. With these restrictions, the Bell-LaPadula model is focused on **maintaining the confidentiality of objects.**

Bell– LaPadula Model

There are three basic properties:

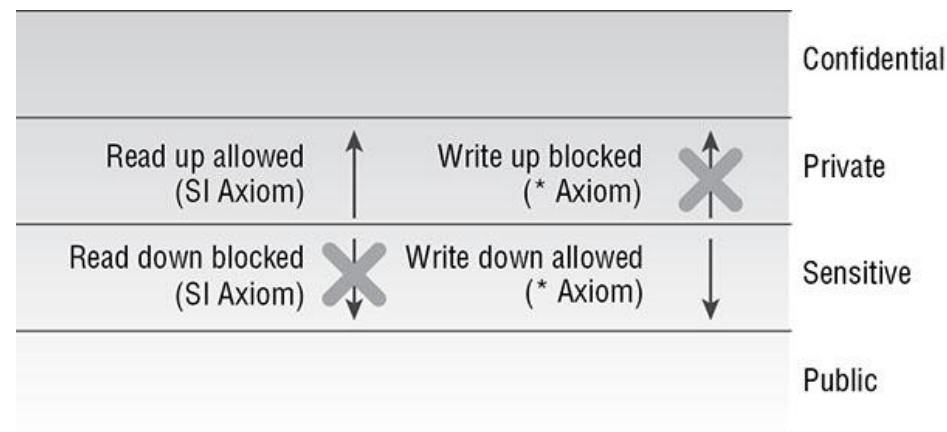
- The *Simple Security Property*
 - a subject may not read information at a higher sensitivity level (no read up)
- The ** (star) Security Property*
 - a subject may not write information to an object at a lower sensitivity level (no write down). This is also known as the *Confinement Property*.
- The *Discretionary Security Property* states that the system uses an access matrix to enforce discretionary access control.

			Secret
Write up allowed (* Property)	↑	Read up blocked (SS Property) 	Classified
Write down blocked (* Property) 		Read down allowed (SS Property) ↓	Sensitive
			Unclassified

Biba Model

The *Biba model* was designed after the Bell-LaPadula model. Where the Bell-LaPadula model addresses confidentiality, the Biba model addresses integrity.

- The *Simple Integrity Property* states that a subject cannot read an object at a lower integrity level (no read-down).
- The ** (star) Integrity Property* states that a subject cannot modify an object at a higher integrity level (no write-up).



Biba Model

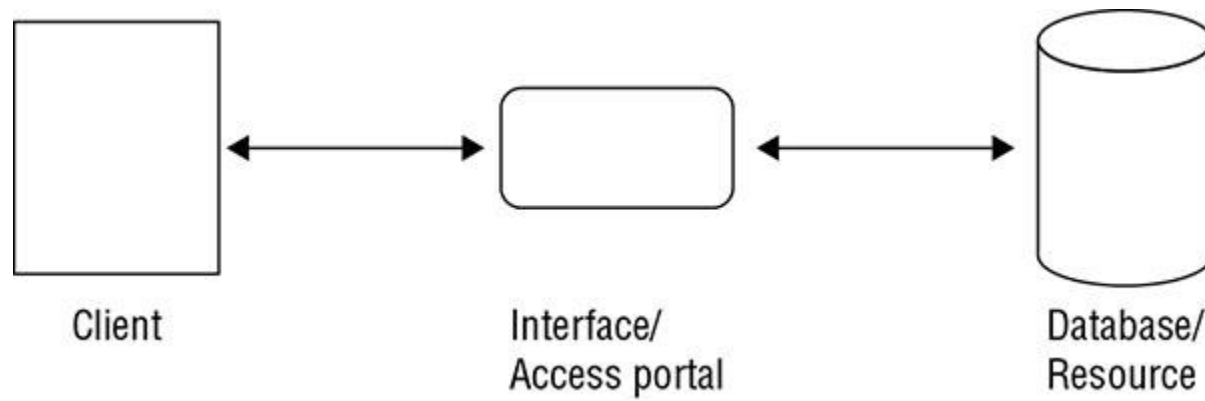
Biba was designed to address three integrity issues:

- Prevent modification of objects by unauthorized subjects.
 - Prevent unauthorized modification of objects by authorized subjects.
 - Protect internal and external object consistency.
-
- A subject cannot write to an object at a higher integrity level.
 - Why can't a subject read an object at a lower integrity level? Think of integrity levels as being like the purity level of air. You would not want to pump air from the smoking section into the clean room environment. The same applies to data. When integrity is important, you do not want unvalidated data read into validated documents. The potential for data contamination is too great to permit such access.

Clark–Wilson Model

- The *Clark-Wilson model* uses a multifaceted approach to enforcing data integrity
 - designed specifically for the commercial environment.
- defines each data item and allows modifications through only a small set of programs.
 - it uses a three-part relationship of subject/program/object known as a *triple* or an *access control triple*.
 - Subjects do not have direct access to objects. Objects can be accessed only through programs. Through the use of two principles—well-formed transactions and separation of duties—the Clark-Wilson model provides an effective means to protect integrity.

Clark–Wilson Model



Brewer and Nash Model (AKA the Chinese Wall model)

- The *Brewer and Nash model* was created to permit access controls to change dynamically based on a user's previous activity
- create security domains that are sensitive to the notion of conflict of interest (for example, someone who works at Company C who has access to proprietary data for Company A should not also be allowed access to similar data for Company B if those two companies compete with each other).
- This model is known as the *Chinese Wall model*
 - it creates a class of data that defines which security domains are potentially in conflict and prevents any subject with access to one domain that belongs to a specific conflict class from accessing any other domain that belongs to the same conflict class.
 - Like putting a wall around all other information in any conflict class.
 - Uses principle of data isolation within each conflict class to keep users out of potential conflict-of-interest situations

Outline

- Principles of Security Models, Design, and Capabilities (CISSP book: ch 8)
 - Secure design principles
 - Techniques for ensuring CIA
 - Fundamentals concepts of security models
 - Understand Security Capabilities of Information Systems
- Security design patterns

Understand Security Capabilities of Information Systems

The security capabilities of information systems include

- Memory protection
- Virtualization
- Trusted Platform Module (TPM)
- Interfaces
- Fault tolerance
- Encryption/Decryption

Memory Protection

- Memory protection is a core security component that must be designed and implemented into an operating system.
- It must be enforced regardless of the programs executing in the system.
- Memory protection is used to prevent an active process from interacting with an area of memory that was not specifically assigned or allocated to it.

Virtualization

- Virtualization technology is used to host one or more operating systems within the memory of a single host computer or to run applications that are not compatible with the host OS.
- Virtualization can be a tool to isolate OSs, test suspicious software, or implement other security protections.

Trusted Platform Module

- The *Trusted Platform Module (TPM)* is both a specification for a cryptoprocessor chip on a mainboard and the general name for implementation of the specification.
- A TPM chip is used to store and process cryptographic keys for the purposes of a hardware supported/implemented hard drive encryption system.
- Generally, a hardware implementation, rather than a software-only implementation of hard drive encryption, is considered to be more secure.

Interfaces

- A *constrained or restricted interface* is implemented within an application to restrict what users can do or see based on their privileges.
 - Users with full privileges have access to all the capabilities of the application.
 - Users with restricted privileges have limited access.
- Applications constrain the interface using different methods.
 - A common method is to hide capability if user doesn't have permissions
 - Commands might be available to administrators via a menu, but if a regular user doesn't have permissions, the command does not appear.
 - Other times, the command is shown but is dimmed or disabled. The regular user can see it but will not be able to use it.
- The purpose of a constrained interface is to limit or restrict the actions of both authorized and unauthorized users.

Fault Tolerance

- *Fault tolerance* is the ability of a system to suffer a fault but continue to operate.
- Fault tolerance is achieved by adding redundant components such as additional disks within a redundant array of inexpensive disks (RAID) array, or additional servers within a failover clustered configuration.
- Fault tolerance is an essential element of security design. It is also considered part of avoiding single points of failure and the implementation of redundancy.

Encryption/Decryption

- Encryption is the process of converting plaintext to ciphertext, whereas decryption reverses that process.
- Symmetric and asymmetric methods of encryption and decryption can be used to support a wide range of security solutions to protect confidentiality and integrity.